

Leave Management System - Microservices Design Document

Document Version: 1.0

Last Updated: June 3, 2026

Table of Contents

1. [Executive Summary](#)
2. [System Architecture](#)
3. [Microservices Overview](#)
4. [Communication Patterns](#)
5. [Technology Stack](#)
6. [Data Architecture](#)
7. [Deployment Architecture](#)
8. [Service Dependencies](#)
9. [API Gateway Configuration](#)
10. [Observability & Monitoring](#)
11. [Security Architecture](#)

Executive Summary

The Leave Management System is a **microservices-based architecture** designed to manage employee leave requests, approvals, and notifications across an organization. The system is built using Python FastAPI services deployed in Docker containers, orchestrated with Docker Compose, and implements modern cloud-native patterns including service discovery, event-driven communication, distributed tracing, and API gateway routing.

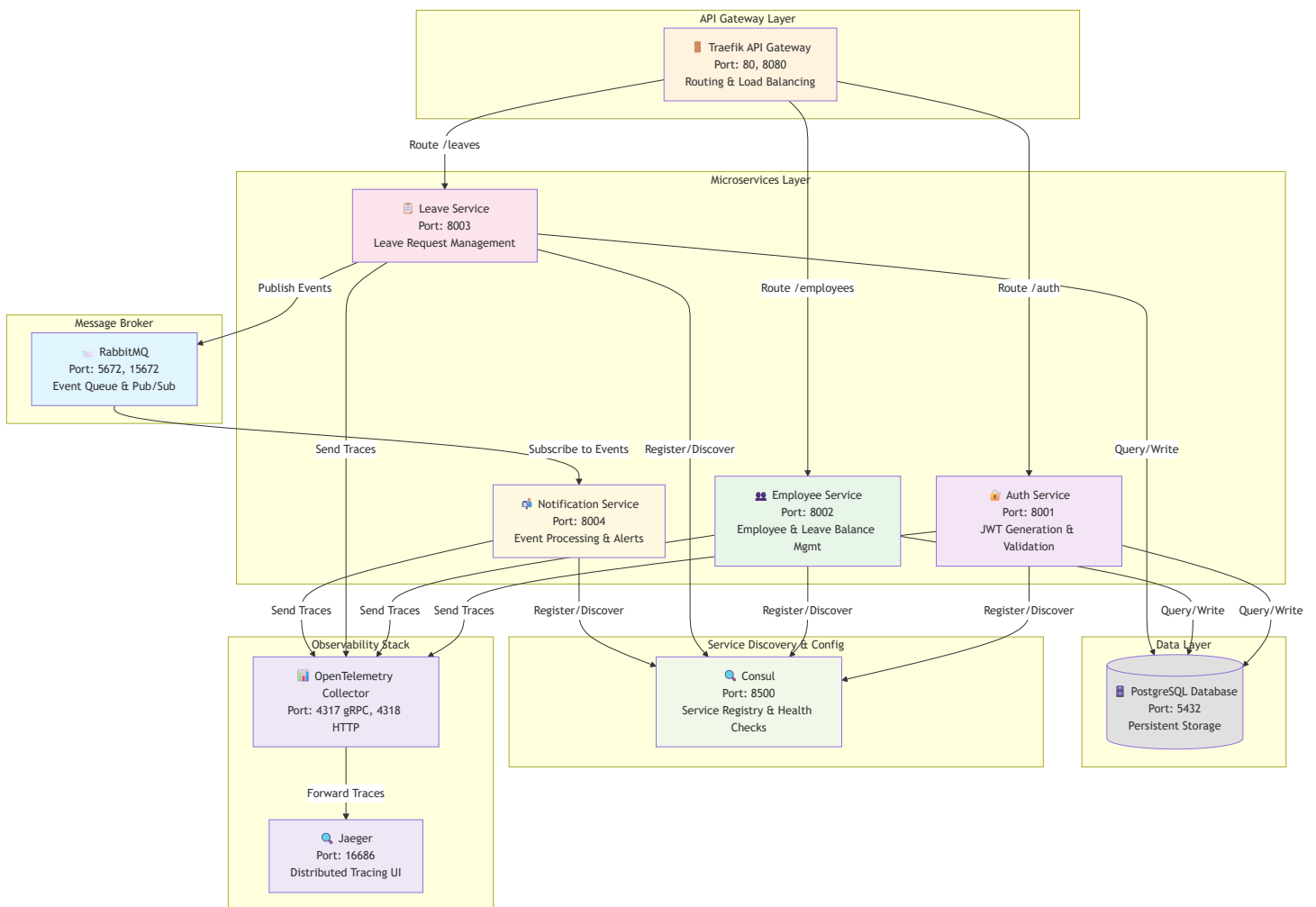
Key Characteristics

- **4 Independent Microservices** for specific business domains
- **Event-Driven Communication** via RabbitMQ for asynchronous operations

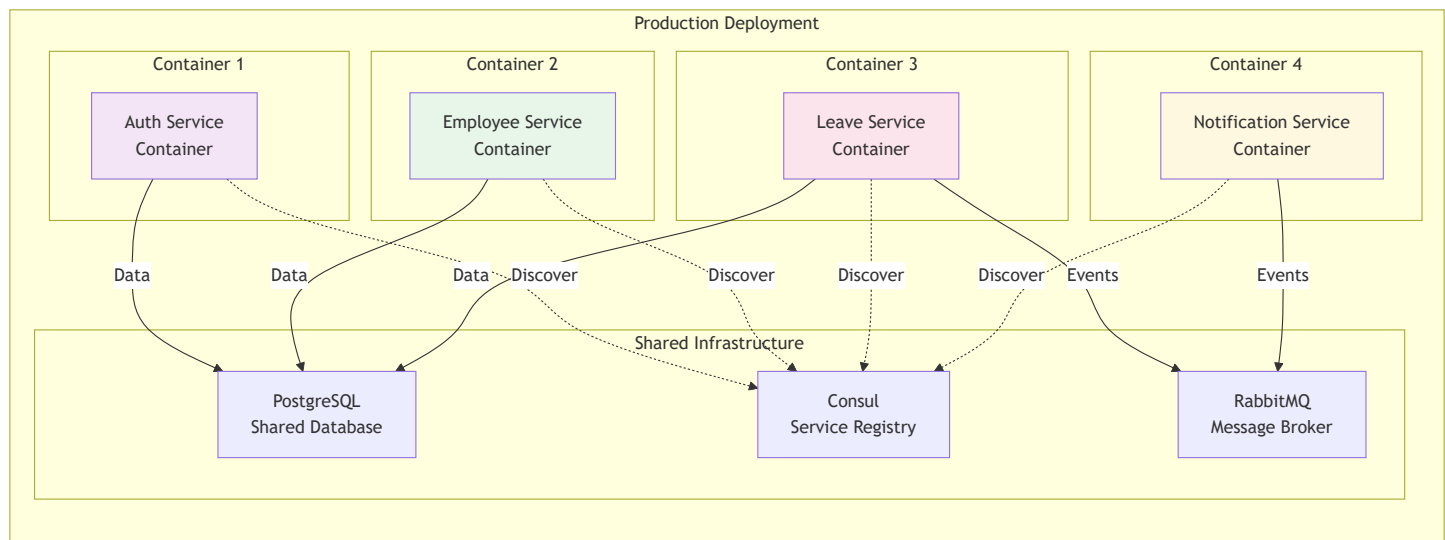
- **Service Discovery** using Consul for dynamic service registration
- **API Gateway** (Traefik) for unified request routing
- **Distributed Tracing** with OpenTelemetry and Jaeger
- **Shared PostgreSQL Database** (note: violates microservices isolation pattern)
- **Docker-based Deployment** with docker-compose orchestration

System Architecture

High-Level Architecture Diagram



Horizontal Service Deployment View



Microservices Overview

1. Auth Service (Port 8001)

Purpose: Handle user authentication, JWT token generation, and validation

Responsibilities:

- User credential validation (username/password)
- JWT token generation and expiration management
- Token validation for other services
- User role assignment (EMPLOYEE, MANAGER)

Key Endpoints:

Method	Endpoint	Purpose
POST	/auth/login	Generate JWT token for authenticated user
GET	/auth/	Service root endpoint

Technology Stack:

- FastAPI (Web framework)
- python-jose (JWT handling)

- passlib + bcrypt (Password hashing)
- SQLAlchemy (ORM)

Database Models:

User

├─ id (PK, unique,)
├─ username (string)
├─ password (string)
├─ role (string)

2. Employee Service (Port 8002)

Purpose: Manage employee information, hierarchy, and leave balances

Responsibilities:

- Store and retrieve employee data
- Maintain employee-manager relationships
- Track and manage leave balances per employee per leave type
- Deduct leave balance when leave is approved
- Provide employee hierarchy queries

Key Endpoints:

Method	Endpoint	Purpose
GET	/employees/me	Get current user employee profile
GET	/employees/leave-balances	Get user's leave balances (all types)
GET	/employees/team/{manager_username}	Get team members for a manager
GET	/employees/{username}/manager	Get employee's reporting manager

Method	Endpoint	Purpose
GET	/employees/{username}/leave-balance/{leave_type}	Get employee's leave balance by leave type
PUT	/employees/{username}/deduct-balance	Deduct leave balance based of particular type
GET	/employees/	Service root endpoint

Technology Stack:

- FastAPI (Web framework)
- SQLAlchemy (ORM)
- PostgreSQL (Database)

Database Models:

Employee

```

├─ id (integer,PK)
├─ username (string, UNIQUE)
├─ full_name (string)
├─ role (string)
├─ manager_id (integer, FK to Employee - nullable)
```

LeaveBalance

```

├─ id (PK)
├─ employee_id (FK to Employee)
├─ leave_type (enum: CASUAL, SICK, EARNED, UNPAID)
├─ allocated (integer)
├─ used (integer)
├─ remaining (integer)
```

3. Leave Service (Port 8003)

Purpose: Manage employee leave requests and manager approvals

Responsibilities:

- Accept leave requests from employees

- Validate leave requests (date ranges, balance availability)
- Route leave for manager approval
- Process manager approvals/rejections
- Update leave balances upon approval
- Publish leave events to message broker
- Detect overlapping leave requests

Key Endpoints:

Method	Endpoint	Purpose
POST	/leaves/apply	Submit a new leave request
GET	/leaves/history	Get user's leave request history
GET	/leaves/pending	Get pending approval leaves (for managers)
PUT	/leaves/{leave_id}/approve	Approve a leave request
PUT	/leaves/{leave_id}/reject	Reject a leave request
GET	/leaves/	Service root endpoint

Technology Stack:

- FastAPI (Web framework)
- SQLAlchemy (ORM)
- PostgreSQL (Database)
- HTTPX (Service-to-service HTTP calls)
- pika (RabbitMQ client)

Database Models:

LeaveRequest

- └─ id (PK, integer)
- └─ employee_username (string)
- └─ leave_type (enum: CASUAL, SICK, EARNED, UNPAID)
- └─ start_date (date)
- └─ end_date (date)
- └─ number_of_days (integer)
- └─ reason (string, max 500 chars)
- └─ status (enum: PENDING, APPROVED, REJECTED)
- └─ manager_comment (string - nullable)
- └─ approved_at (timestamp - nullable)
- └─ rejected_at (timestamp - nullable)

Constraints:

- Unique(employee_username, start_date, end_date)

4. Notification Service (Port 8004)

Purpose: Consume leave events and trigger notifications

Responsibilities:

- Listen for leave-related events on RabbitMQ
- Process leave.applied events (notify manager)
- Process leave.approved events (notify employee)
- Process leave.rejected events (notify employee)
- Log and track notifications
- Handle event processing errors

Events Consumed:

Event	Triggered By	Payload
leave.applied	Leave Service	employee_username, leave_type, dates, reason
leave.approved	Leave Service	employee_username, manager_username, leave_type
leave.rejected	Leave Service	employee_username, manager_comment

Technology Stack:

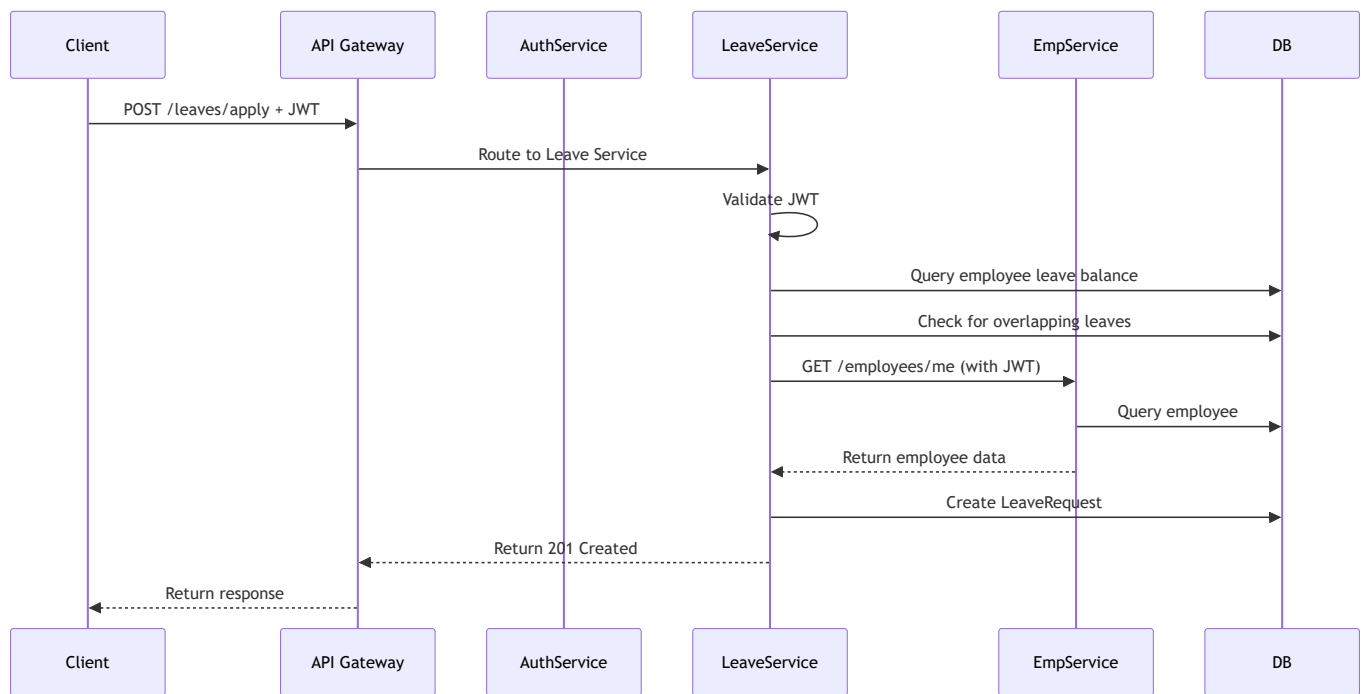
- FastAPI (Web framework)
- pika (RabbitMQ client)
- OpenTelemetry (Distributed tracing)

Communication Patterns

Synchronous Communication (Request-Response)

Mechanism: HTTP/REST with Bearer Token Authentication

Diagram:



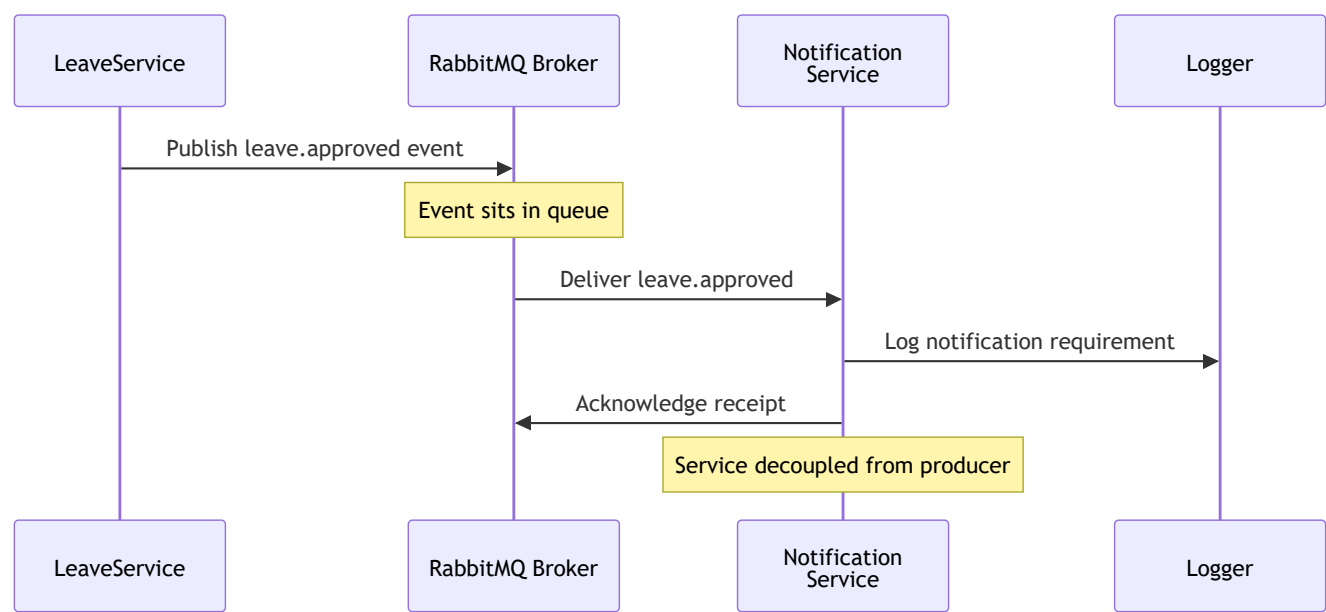
Service-to-Service Communication:

- **Leave Service → Employee Service:**
 - Query employee leave balances
 - Deduct leave balance on approval
 - Get employee manager info
- **All Services → Consul:** Service discovery queries

Asynchronous Communication (Event-Driven)

Mechanism: RabbitMQ TOPIC Exchange

Diagram:



Event Flow:

- 1. Leave Service creates leave request → Publishes `leave.applied` event
- 2. Leave Service approves leave → Publishes `leave.approved` event
- 3. Leave Service rejects leave → Publishes `leave.rejected` event
- 4. Notification Service subscribes to all `leave.*` events
- 5. Notification Service processes events independently

Technology Stack

Application Layer

Technology	Purpose	Version
Python	Programming language	3.12+
FastAPI	Web framework	Latest
Uvicorn	ASGI server	Latest
SQLAlchemy	ORM framework	2.0+
Pydantic	Data validation	2.0+

Authentication & Security

Technology	Purpose
python-jose	JWT handling
passlib	Password hashing library
bcrypt	Password hashing algorithm
FastAPI Security	Bearer token validation

Data & Storage

Technology	Purpose
PostgreSQL	Relational database
psycopg2	PostgreSQL Python adapter

Inter-Service Communication

Technology	Purpose
HTTPX	Async HTTP client
RabbitMQ	Message broker
pika	RabbitMQ Python client

Service Architecture

Technology	Purpose
Consul	Service registry & discovery
Traefik	API Gateway & reverse proxy

Monitoring & Observability

Technology	Purpose
OpenTelemetry	Distributed tracing SDK

Technology	Purpose
Jaeger	Tracing backend & UI
OpenTelemetry Collector	Telemetry data processor

Deployment & Orchestration

Technology	Purpose
Docker	Container runtime
Docker Compose	Container orchestration

Data Architecture

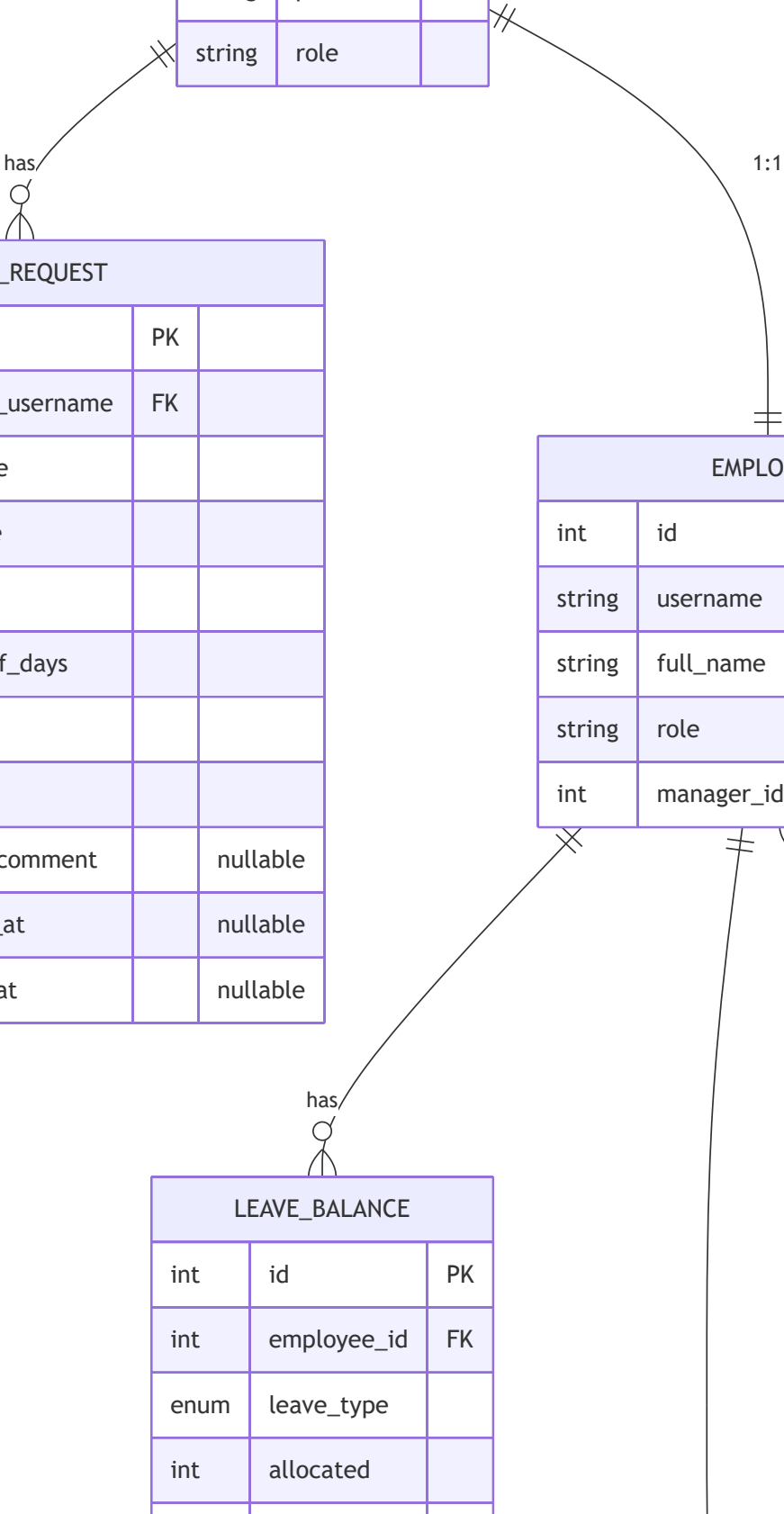
Database Schema Overview

USER		
int	id	PK
string	username	UK
string	password	
string	role	

LEAVE_REQUEST			
int	id	PK	
string	employee_username	FK	
enum	leave_type		
date	start_date		
date	end_date		
int	number_of_days		
string	reason		
enum	status		
string	manager_comment		nullable
timestamp	approved_at		nullable
timestamp	rejected_at		nullable

EMPLOYEE			
int	id	PK	
string	username	UK	
string	full_name		
string	role		
int	manager_id	FK	nullable

LEAVE_BALANCE		
int	id	PK
int	employee_id	FK
enum	leave_type	
int	allocated	



int	used	
int	remaining	



Shared Data Attributes

Leave Types (Enum - Shared):

- CASUAL : General casual leave
- SICK : Medical/sick leave
- EARNED : Earned/annual leave
- UNPAID : Unpaid leave

Leave Status (Enum - Shared):

- PENDING : Awaiting manager approval
- APPROVED : Approved by manager
- REJECTED : Rejected by manager

User Roles (Enum - Shared):

- EMPLOYEE : Regular employee
- MANAGER : Manager with approval authority

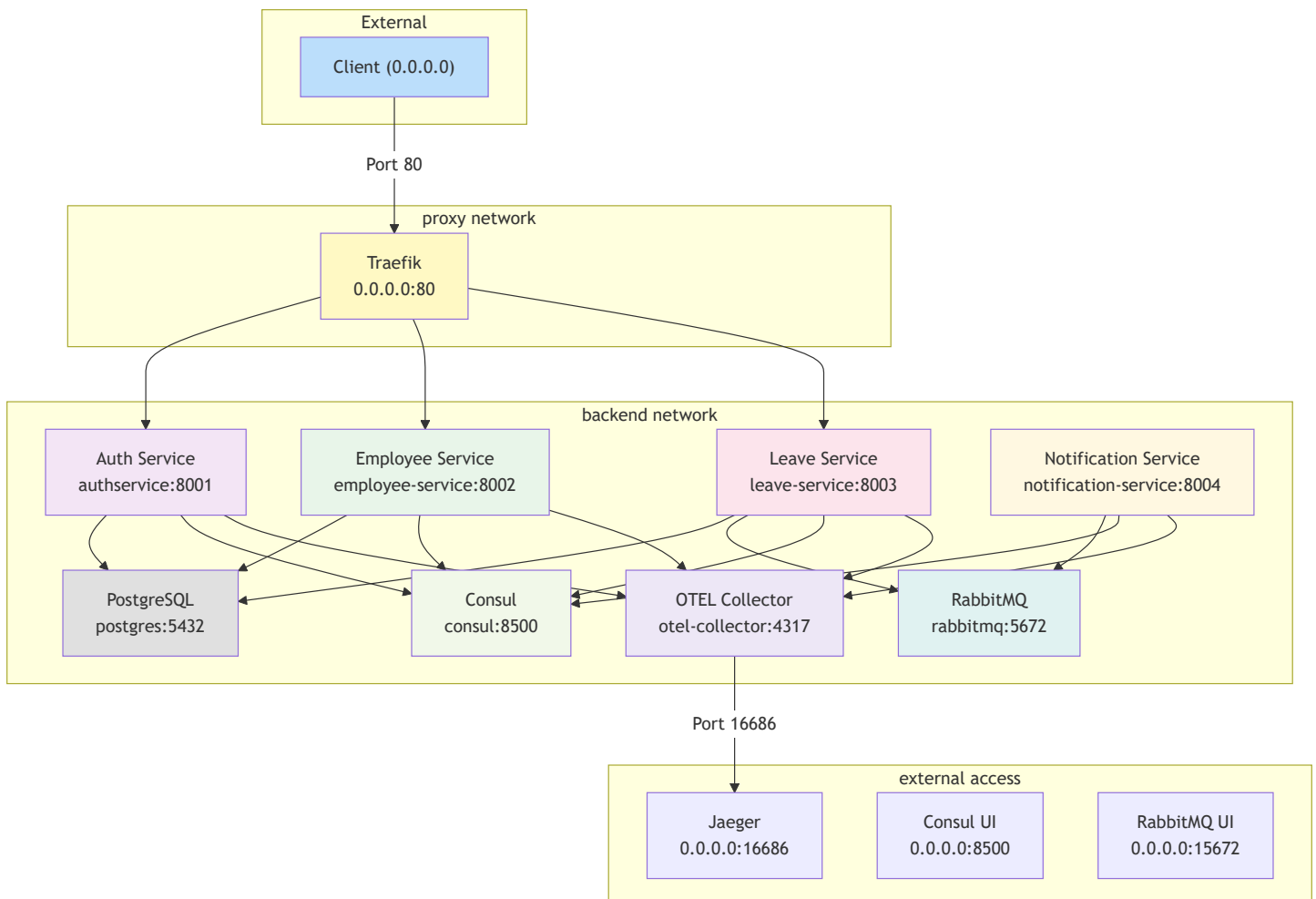
Deployment Architecture

Docker Compose Services

Services in docker-compose.yml:

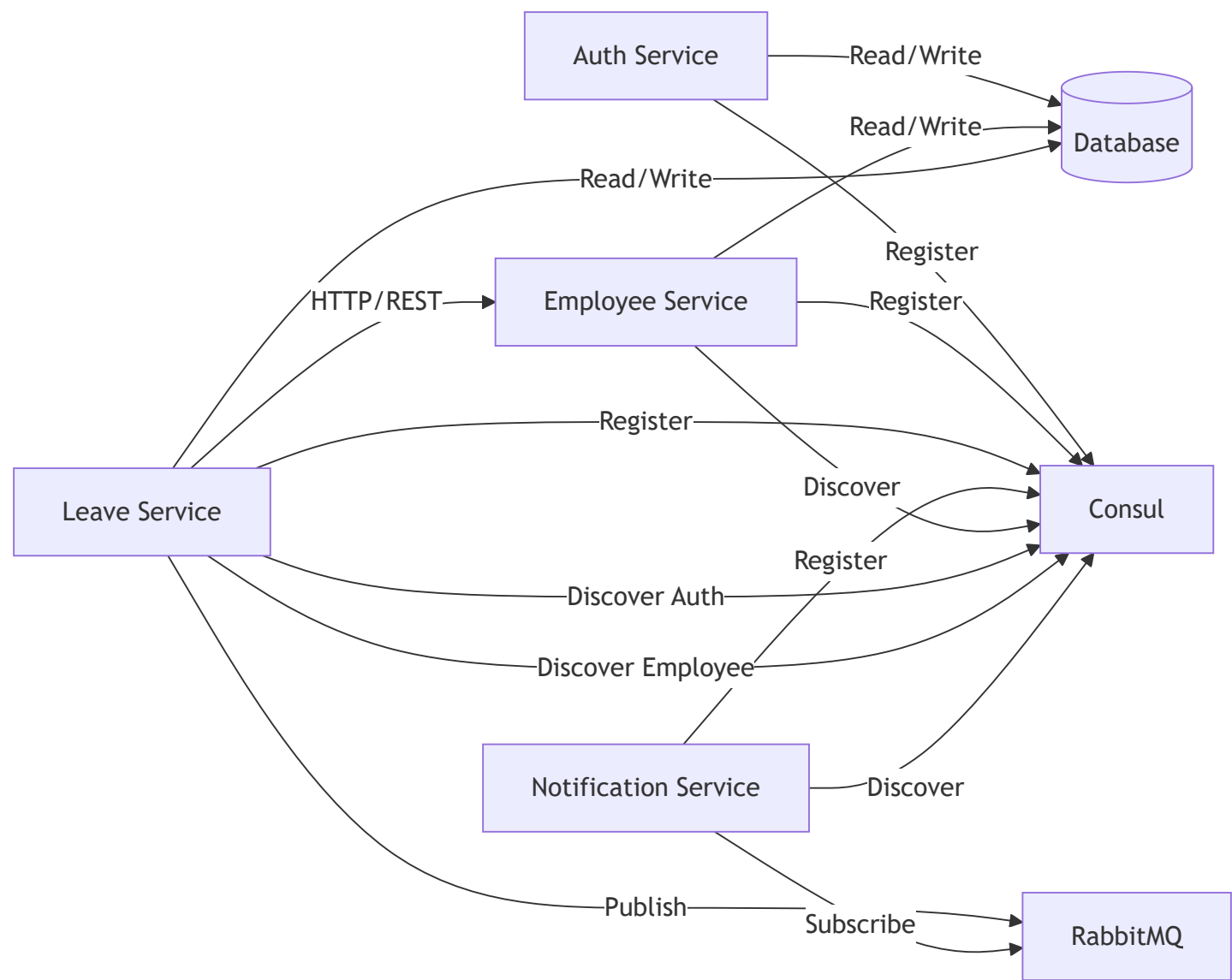
- └─ traefik (API Gateway)
- └─ consul (Service Discovery)
- └─ postgres (Database)
- └─ rabbitmq (Message Broker)
- └─ otel-collector (Telemetry Collector)
- └─ jaeger (Tracing Backend)
- └─ auth-service (Python/FastAPI)
- └─ employee-service (Python/FastAPI)
- └─ leave-service (Python/FastAPI)
- └─ notification-service (Python/FastAPI)

Network Architecture



Service Dependencies

Dependency Matrix



Call Dependencies

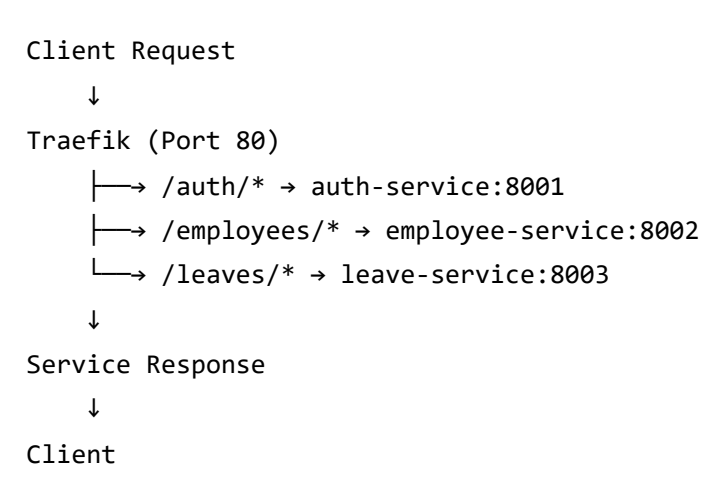
Source	Target	Method	Purpose
Leave Service	Auth Service	HTTP	Validate JWT token
Leave Service	Employee Service	GET /employees/me	Get current employee info

Source	Target	Method	Purpose
Leave Service	Employee Service	GET /employees/{username}/leave-balance/{type}	Check leave balance
Leave Service	Employee Service	PUT /employees/{username}/deduct-balance	Deduct balance on approval
Notification Service	RabbitMQ	SUBSCRIBE	Listen for leave events

API Gateway Configuration

Traefik Routing Rules

Request Flow:



Router Definitions (in traefik/dynamic.yml):

auth-router:

- rule: Path(/auth/*)
- service: auth-service
- port: 8001

employee-router:

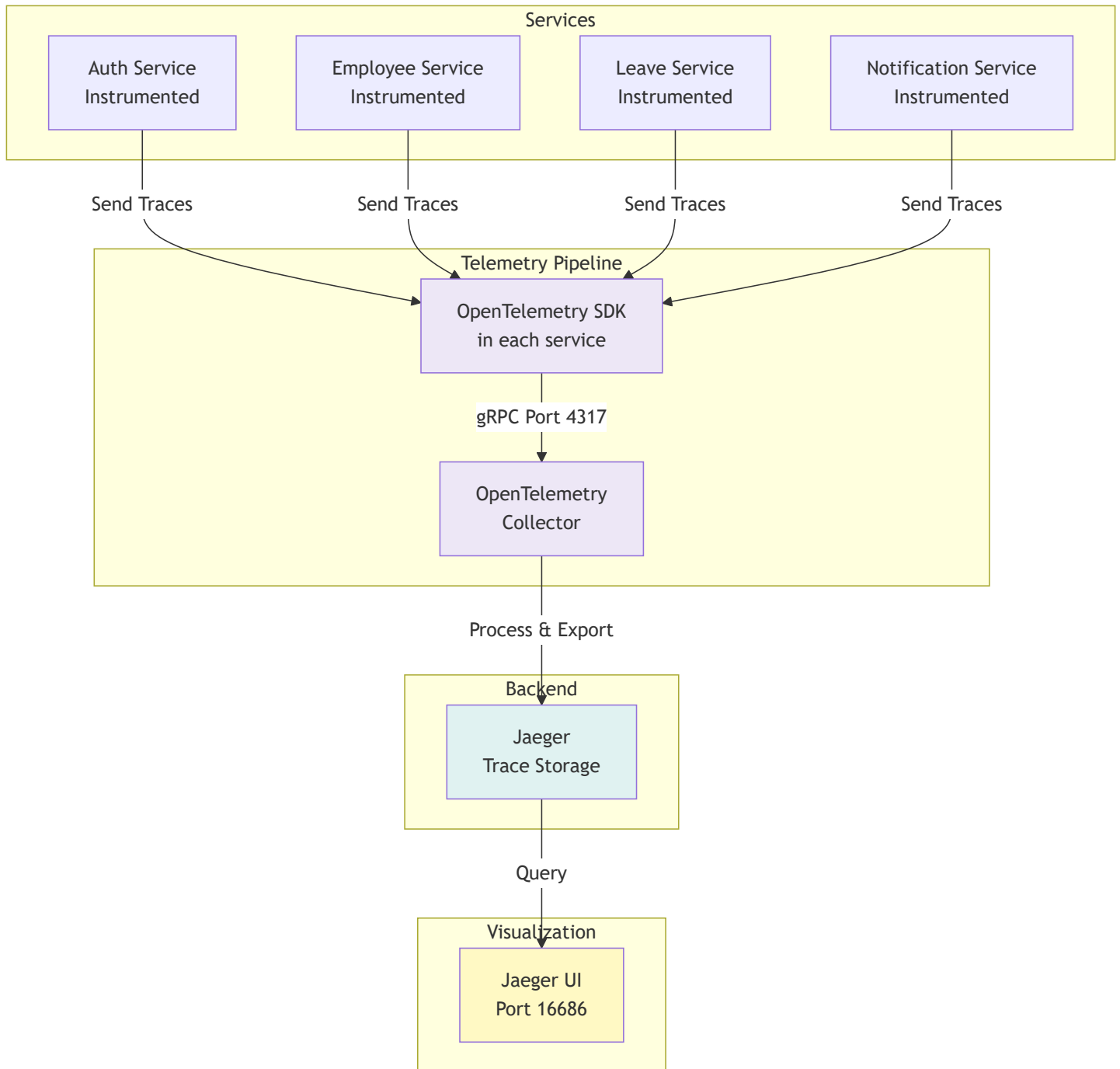
- rule: Path(/employees/*)
- service: employee-service
- port: 8002

leave-router:

- rule: Path(/leaves/*)
- service: leave-service
- port: 8003

Observability & Monitoring

Distributed Tracing with OpenTelemetry



Health Checks

Service Health Endpoints:

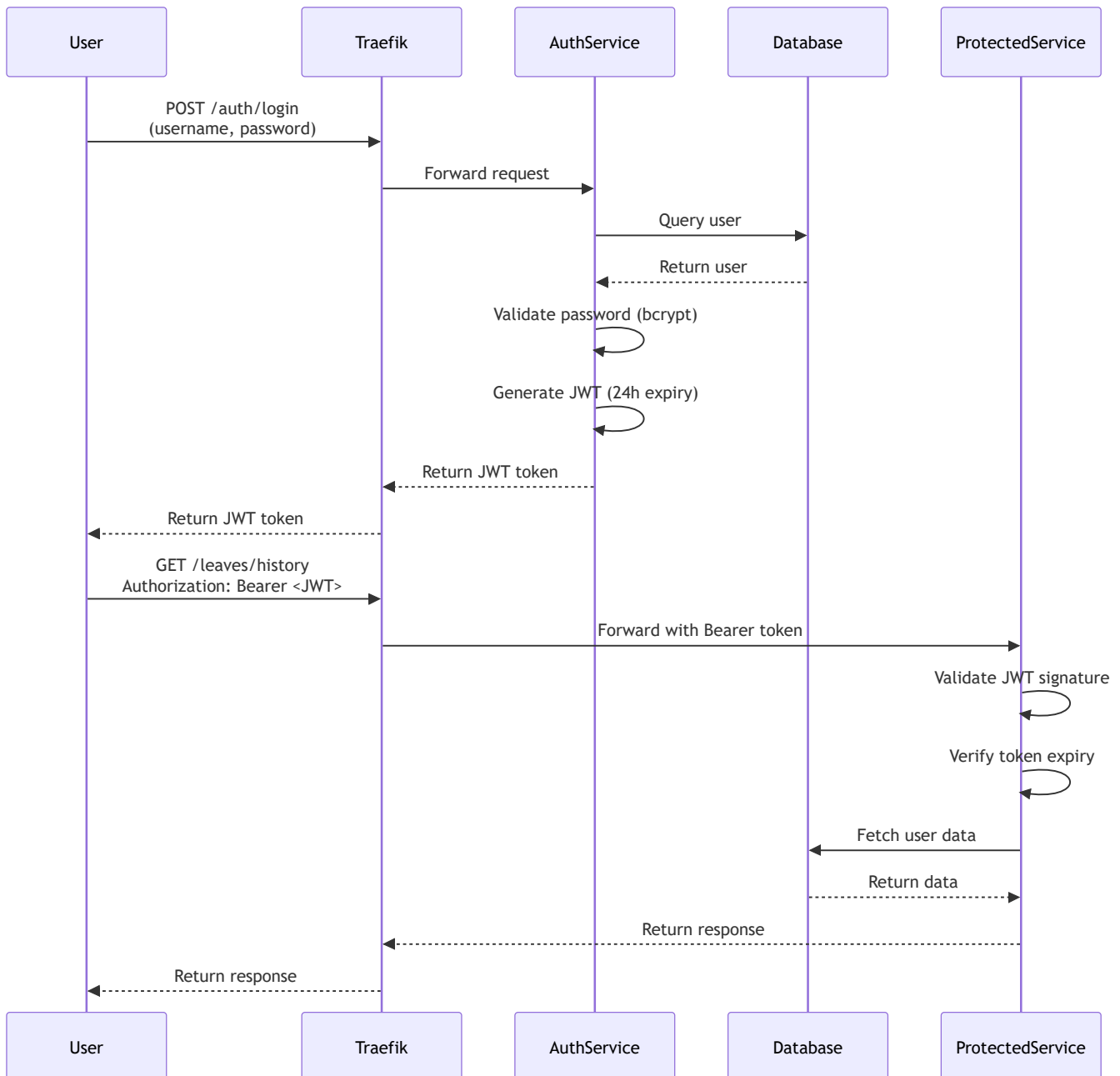
- Auth Service: GET /health
- Employee Service: GET /health

- Leave Service: GET /health
- Notification Service: Starting/shutdown checks

Consul Health Checks: Each service registers with Consul for health monitoring

Security Architecture

Authentication Flow



Token Security

- **Algorithm:** HS256 (HMAC with SHA-256)
- **Expiry:** 1440 minutes (24 hours)
- **Signature:** Uses SECRET_KEY
- **Validation:** Performed on every protected route

Role-Based Access Control (RBAC)

User Roles:

```
|— EMPLOYEE
|   |— Can apply for leave
|   |— Can view own leave history
|   |— Can view own leave balances
|— MANAGER
|   |— Can approve/reject team member leaves
|   |— Can view team leave ``
```

Password Security

- **Algorithm**: Bcrypt with salt
- **Hashing**: Stored in `User.hash\_password`
- **Validation**: passlib library handles comparison

Deployment Steps

Prerequisites

- Docker & Docker Compose installed
- Python 3.12+ (for local development)
- PostgreSQL client tools (optional)

Startup Procedure

```
```bash
```

```
1. Navigate to project root {#1-navigate-to-project-root }
```

```
cd /path/to/leave_management
```

```
2. Start all services with docker-compose {#2-start-all-services-with-docker-compose }
```

```
docker-compose up -d
```

```
3. Verify services are running {#3-verify-services-are-running }
```

```
docker-compose ps
```

```
4. Access services {#4-access-services }
```

```
API Gateway: http://localhost/ {#api-gateway-httplocalhost }
```

```
Jaeger UI: http://localhost:16686 {#jaeger-ui-httplocalhost16686 }
```

```
Consul UI: http://localhost:8500 {#consul-ui-httplocalhost8500 }
RabbitMQ UI: http://localhost:15672 (guest:guest) {#rabbitmq-ui-httplocalhost15672-guestguest
```

## Shutdown Procedure

```
Stop all services {#stop-all-services }
docker-compose down

Stop and remove volumes (careful!) {#stop-and-remove-volumes-careful }
docker-compose down -v
```